Website User Guide

A friendly handbook for contributing content to publicai.ch

Public AI Switzerland

2 November 2026

Contents

1 Welcome 5

2 The big picture 7

3 Run a local preview 9

4 Edit an existing page 11

5 Create a new page 13

6 Work in five languages 15

7 Format text and add images 17

8 Publish a news article 19

9 Menus and footer 21

10 Next steps 23

Welcome

This handbook shows you how to update the Public AI Switzerland website: editing pages, publishing news, and changing the navigation. You do not need to know React or any programming language — everyday content work happens in plain text files and one configuration file.

The website speaks five languages: English (at the root address), German, French, Italian, and Romansh. Each page keeps all of its translations in a single file, so you can always see which languages are still missing.

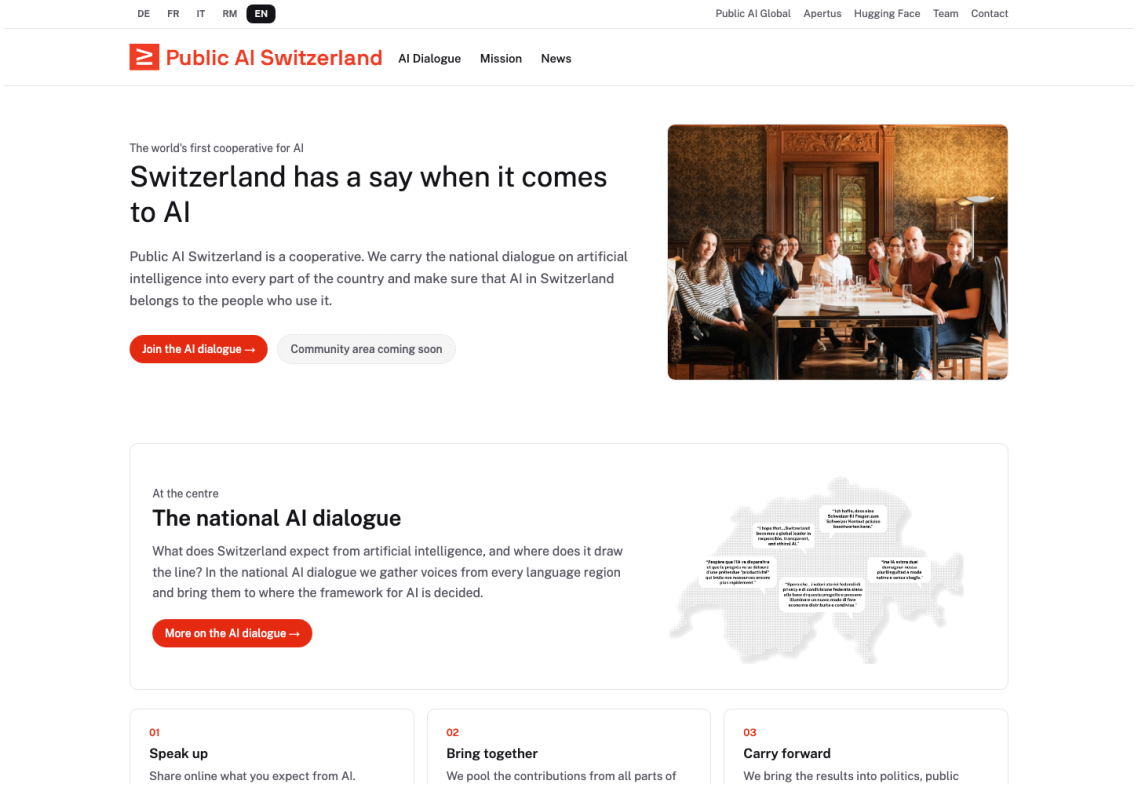


Figure 1.1: The English home page in a local preview.

The big picture

Three places cover almost all everyday work:

- `content/` — the text of every page, in MDX files (Markdown plus optional layout blocks).
- `public/assets/` — images and downloadable files.
- `config.json` — the header menus, footer links, news categories, redirects, and a few switches.

Everything else is generated. The build turns your MDX files into static pages, rebuilds the sitemap, checks every internal link, and refuses to publish a page that nothing links to.

To find the file behind an address, drop the slashes: `/mission/` comes from `content/cooperative/mission` and the German page is `/de/mission/`. News addresses include the date: `/news/2026-09-24-zurich-hackath`

Run a local preview

A preview on your own machine lets you check changes before anyone else sees them.

```
git clone https://github.com/forpublicai/publicai.ch.git
cd publicai.ch
corepack enable
yarn install --immutable
yarn dev
```

Open the printed address, usually **`http://localhost:5173/`**. Save an edit and look at the browser. Stop the server with Ctrl+C.

Restart **`yarn dev`** whenever you add or delete a file, add or remove a language section, or change a page's **`draft`** status. Plain wording edits refresh on their own.

Edit an existing page

1. Find the file whose name matches the address — for `/mission/` that is `content/cooperative/mission.m`
2. Find the language section you want, for example `<!-- locale: en -->`.
3. Edit the text below that marker, stopping at the next marker.
4. If the heading or summary changed, update that language's `title` and `description` in the frontmatter at the top.
5. Save, check the preview, and run `yarn build` before handing over the change.

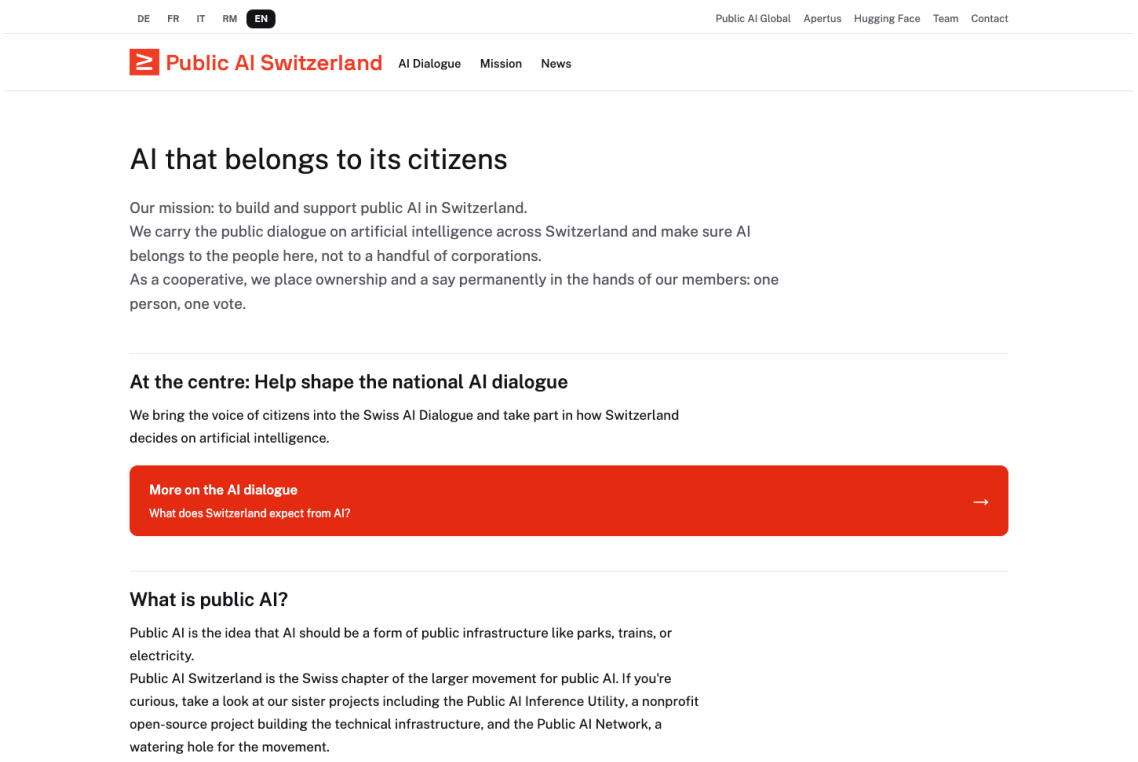


Figure 4.1: The Mission page, rendered from its MDX file.

An MDX file has two parts: the frontmatter between the `---` lines (title, description, flags), and the body divided by language markers. Keep both in place.

Create a new page

Copy the template and give the copy a unique, stable filename (lowercase letters, numbers, hyphens; avoid the reserved names **news** and **authors**):

```
cp content/templates/page-template.mdx content/community/workshops.mdx
```

Delete the template's instruction paragraph before the first language marker — no body text may sit above a marker. Then write your content under the markers, one per language.

A minimal English-only page looks like this:

```
---
title: Workshops
description: Meet the community and take part in our public AI workshops.
---

<!-- locale: en -->

Come and learn with us.

## What to expect

[Read our mission](/mission)
```

This creates **/workshops/** in English only. Add German, French, Italian, and Romansh sections to create **/de/workshops/** and so on.

Link the new page from an existing page or from **config.json**, otherwise the build's reachability check fails. While drafting, set **draft: true**; drafts have no routes at all, even locally.

Work in five languages

The language codes are `en`, `de`, `fr`, `it`, and `rm`. Each marker goes on its own line, each language appears at most once, and at least one section is required.

Translations are written by the author, not generated automatically. That is deliberate: legal text such as the statutes must not be machine-translated, and every translation should be reviewed by a person. When a fact, date, or link changes, review the other language sections too.

Translate the metadata as well:

```
title: Workshops
title_fr: Ateliers
description: Discover our workshops and join the next gathering.
description_de: Entdecken Sie unsere Workshops und das naechste Treffen.
```

A missing `title_xx` falls back to English, then German, then the base field — so set explicit overrides for each finished translation.

Only languages with a section get a page route. Requesting a missing translation can produce a 404 rather than a fallback, so click through every language when reviewing. The build writes `untranslated-sections-report.txt` listing any gaps.

Format text and add images

Ordinary Markdown works everywhere: headings with `##`, bold, lists, and links. Internal links such as `/mission` automatically carry the current language prefix.

Save images under `public/assets/` and reference them from `/assets/`:

```
<Figure
  src="/assets/images/workshops/workshop.jpg"
  alt="Participants discussing ideas around a workshop table"
  caption="A community workshop in Zurich."
/>
```

Write alt text that conveys the useful information, and translate captions and alt text per language.

Ready-made blocks cover common layouts:

```
<Lead>A short introduction that stands out.</Lead>
<Callout variant="info">Registration details will be shared soon.</Callout>
<CTA title="Read our mission" to="mission" />
```

`to="..."` takes a page slug; external buttons use `href="https://..."`. The full catalogue — stat grids, accordions, team cards, poster downloads, video embeds — is in `docs/reference/components.md`. Embeds only accept hosts listed in `embeds.allowlist` in `config.json`.

Publish a news article

A news article is an MDX file whose frontmatter has both `date` and `category`.

```
cp content/templates/news-template.mdx content/news/2026-11-09-community-workshop.mdx
```

Use the publication date at the start of the filename and in the frontmatter. The category must be one of `news.categories` in `config.json`: `Dialog`, `Genossenschaft`, `Community`, `Medien`, `Technik`.

```
---
title: A community workshop
description: Our community explored public AI and planned the next steps.
date: 2026-11-09
category: Community
---

<!-- locale: en -->

We met to exchange ideas and learn together.
```

The article then appears automatically at `/news/2026-11-09-community-workshop/`, in the News listing (newest first), and in the home teaser (latest three). Set `draft: true` to hide it while working, and use `externalUrl` when the article really lives on another site.

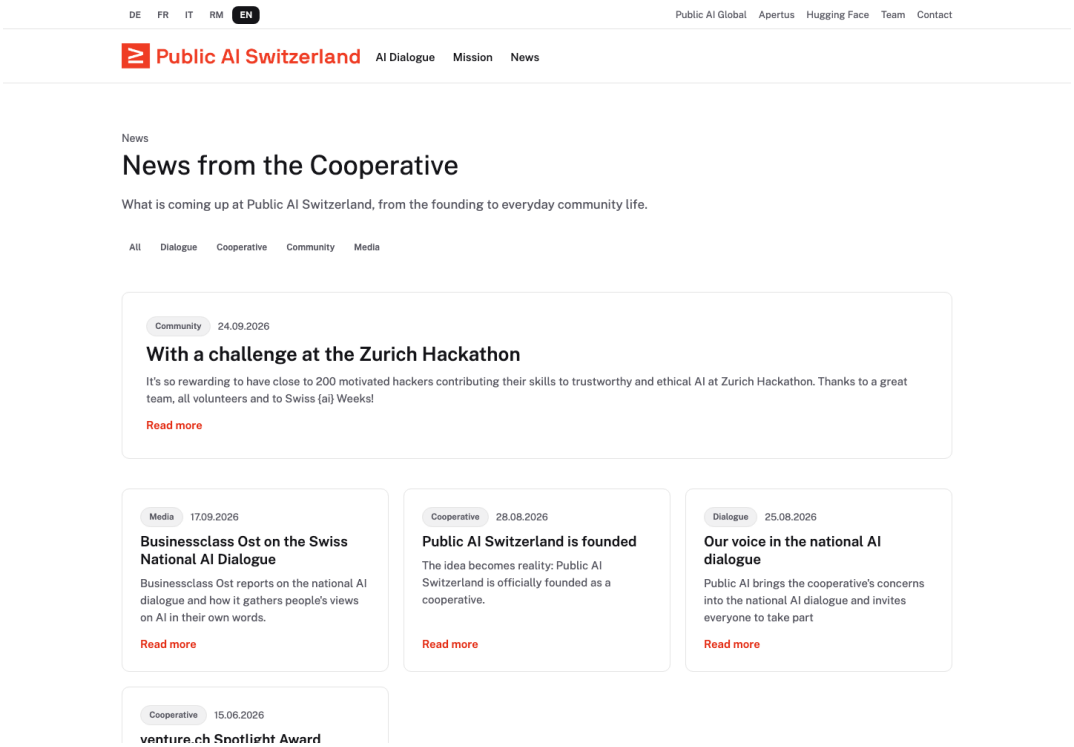


Figure 8.1: The generated News listing.

Menus and footer

All navigation lives in `config.json`. Each menu item needs a five-language `label` and exactly one target:

- `"page": "mission"` — an internal page slug;
- `"url": "https://publicai.co"` — an external or `mailto:` link;
- `"children": [...]` — a one-level dropdown of further items.

```
{
  "label": { "en": "Workshops", "de": "Workshops", "fr": "Ateliers",
             "it": "Workshop", "rm": "Lavuratoris" },
  "page": "workshops"
}
```

Keep JSON punctuation intact (double quotes, commas between entries, none after the last). The build fails with the exact config path if a label is missing a language or a `page` slug does not exist. The footer works the same way, including a compact newsletter sign-up form below the tagline; check narrow screens too, since both menus fold into one mobile menu.

Next steps

Before handing over a change, run the full check chain:

```
yarn lint
yarn typecheck
yarn test
yarn build
yarn test:e2e # needs Chromium: yarn playwright install chromium
```

Then open a preview of the built site with `yarn build && yarn preview` (usually `http://localhost:4173/`). The live website only changes when a new build is deployed, and only `build/client/` is ever uploaded — never commit generated output.

For details, see the how-to guides and references in `docs/`, the illustrated `docs/pdf/quick-guide-for-cont` and `docs/deploy/server-admin-guide.md` for deployment.